01 Scrum-Based Requirements Collection Template

Student template for collecting and managing project requirements

Field	Description
Course	Informatics for Telecommunications
Program	Telecommunication Engineering
Project	Centralized Routing System
Team name	The developers.
Team members	Diana Isabel Chamorro Daza Juan Jose Herrera Cruz Jose David Cometa
Professor	Oscar Mauricio Caicedo Rendon
Date	27/04/2026
Version	1.0

1. Product Vision

The purpose of this project is to develop a software-based centralized routing system where router applications communicate with a controller application. The controller receives topology information, computes routes, generates routing tables, and distributes them to routers.

2. Product Goal

Develop a centralized routing system in Python composed of a controller and multiple router applications. The controller must compute routing tables from the network topology and send them to the router applications using TCP or UDP and JSON messages.

3. Stakeholders

Stakeholder	Role	Interest
Professor	Evaluator and project guide	Verify correct application of programming, software engineering, and networking concepts
Students	Developers, Testers, Documenters	Design, implement, test, and demonstrate the system
Controller application	System component	Compute and distribute routing information
Router application	System component	Send local topology information and receive routing tables
Network administrator	User	Monitor the network and validate routing behavior

4. User Story Format

Use the following format for each requirement:

As a [type of user], I want [functionality], so that [benefit].

5. Functional Requirements Backlog

ID	User story	Priority	Acceptance criteria	Sprint	Status
FR-01	As a router application, I want to register with the controller so that the controller can identify me in the network.	High	The router sends ID, IP, and port. The controller stores the router information.	Sprint 1	Passed
FR-02	As a router application, I want to send neighbor information to the controller so that the controller can build the network topology.	High	The router sends neighbors and link costs. The controller updates the topology.	Sprint 1	Passed
FR-03	As a controller, I want to store the network topology so that routing algorithms can be executed.	High	The topology is stored using a graph, dictionary, adjacency list, or JSON file.	Sprint 1	Passed
FR-04	As a controller, I want to compute shortest paths so that optimized routing tables can be generated.	High	The controller computes best paths from each router to every destination.	Sprint 2	Passed
FR-05	As a controller, I want to generate a routing table for each router so that each router knows the next hop for every destination.	High	Each table contains destination, next hop, and cost.	Sprint 2	Passed
FR-06	As a router application, I want to receive my routing table from the controller so that I can make forwarding decisions.	High	The router receives and stores its routing table locally.	Sprint 2	Passed
FR-07	As a router application, I want to display my routing table so that the administrator can verify the routing information.	Medium	The routing table is shown in the command-line interface.	Sprint 2	Passed
FR-08	As a network administrator, I want to simulate a link cost change so that I can observe route recalculation.	Medium	The controller recalculates affected routing tables after the update.	Sprint 3	Passed
FR-09	As a controller, I want to log important events so that system	Low	Registration, topology updates, route computation,	Sprint 3	Passed

	execution can be analyzed.		and table delivery are logged.		

6. Non-Functional Requirements Backlog

ID	Requirement	Description	Priority	Acceptance criteria
NFR-01	Python implementation	The system must be implemented in Python.	High	All main applications are Python programs.
NFR-02	Modularity	Separate controller logic, router logic, communication logic, and routing algorithm logic.	High	Code is organized into modules, classes, or packages.
NFR-03	Command-line execution	The system must be operated from the terminal.	High	Controller and routers can be started from the command line.
NFR-04	JSON messages	Messages between routers and controller must use JSON.	High	All exchanged messages are valid JSON objects.
NFR-05	Reliability	The system should handle invalid messages without crashing.	Medium	Invalid messages generate error responses or logs.
NFR-06	Scalability	The system should support at least four router applications.	Medium	The controller manages four or more routers correctly.
NFR-07	Maintainability	The code should be readable and documented.	Medium	Functions and classes include meaningful names and comments.
NFR-08	Testability	The system should include tests for the main requirements.	Medium	Test cases are documented and executed.

7. Product Backlog

Backlog ID	Requirement ID	Task	Responsible student	Estimated effort	Sprint	Status
PB-01	FR-00	Implement router registration messagex	Jose Cometa	3 h	Sprint 1	Completed
PB-02	FR-01	Implement controller router registry	Juan Herrera	3 h	Sprint 1	Completed
PB-03	FR-02	Define JSON format for neighbor information	Juan Herrera	2 h	Sprint 1	Completed
PB-04	FR-03	Implement topology data structure	Diana Chamorro	4 h	Sprint 1	Completed
PB-05	FR-04	Implement Dijkstra algorithm	Jose Cometa	5 h	Sprint 2	Completed
PB-06	FR-05	Generate routing table for each router	Jose Cometa	4 h	Sprint 2	Completed
PB-07	FR-06	Send routing table from	Juan Herrera	4 h	Sprint 2	Completed

		controller to router				
PB-08	FR-07	Display routing table in router CLI	Diana Chamorro	2 h	Sprint 2	Completed
PB-09	FR-08	Simulate link cost change	Diana Chamorro	4 h	Sprint 3	Completed
PB-10	FR-09	Add event logging	Juan Herrera	2 h	Sprint 3	Completed

8. Sprint Planning Template

Field	Description
Sprint number	Sprint 1
Sprint goal	Build the communication foundation between router applications and the central controller, allowing router registration and network topology collection.
Start date	05/05/2026
End date	12/05/2026
Team members	Diana Chamorro, Juan Herrera, José Cometa

Task ID	Requirement ID	Task description	Responsible	Estimated time	Status
T-01	FR-00	Implement router registration messagex	Jose Cometa	3 h	Completed
T-02	FR-01	Implement controller router registry	Juan Herrera	3 h	Completed
T-03	FR-02	Define JSON format for neighbor information	Juan Herrera	2 h	Completed
T-04	FR-03	Implement topology data structure	Diana Chamorro	4 h	Completed

Field	Description
Sprint number	Sprint 2
Sprint goal	Process the collected topology to compute shortest paths and automatically generate routing information for each router.
Start date	13/05/2026
End date	20/05/2026
Team members	Diana Chamorro, Juan Herrera, José Cometa

Task ID	Requirement ID	Task description	Responsible	Estimated time	Status
T-01	FR-04	Implement Dijkstra algorithm	Jose Cometa	5 h	Completed
T-02	FR-05	Generate routing table for each router	Jose Cometa	4 h	Completed

T-03	FR-06	Send routing table from controller to router	Juan Herrera	4 h	Completed
T-04	FR-07	Display routing table in router CLI	Diana Chamorro	2 h	Completed

Field	Description
Sprint number	Sprint 3
Sprint goal	Validate the routing system under topology changes, register operational events, and perform final integration testing.
Start date	26/05/2026
End date	06/06/2026
Team members	Diana Chamorro, Juan Herrerax

Task ID	Requirement ID	Task description	Responsible	Estimated time	Status
T-01	FR-08	Simulate link cost change	Diana Chamorro	4 h	Completed
T-02	FR-09	Add event logging	Juan Herrera	2 h	Completed

9. Definition of Done

Criterion	Completed?
The functionality is implemented in Python	Completed
The code was tested successfully	Completed
Expected input and output were documented	Completed
Acceptance criteria were met	Completed
The code was integrated with the rest of the system	Completed
The functionality was demonstrated to the professor	Completed

02 Requirements Test Plan Template

Student template for validating functional and non-functional requirements

Field	Description
Course	Informatics for Telecommunications
Program	Telecommunication Engineering
Project	Centralized Routing System
Team name	The developers.
Team members	Juan Jose Herrera, Jose David Cometa, Diana Isabel Chamorro
Professor	Oscar Mauricio Caicedo Rendon
Date	15/06/2026
Version	1.0

1. Test Plan Objective

The objective of this test plan is to verify that the centralized routing system satisfies the defined requirements. The tests must validate router registration, topology collection, routing algorithm execution, routing table generation, communication between controller and routers, and system behavior under different network scenarios.

2. Testing Scope

Included in testing	Status
Controller application	Yes
Router application	Yes
Communication protocol	Yes
JSON message format	Yes
Topology storage	Yes
Routing algorithm	Yes
Routing table generation	Yes
Command-line interface	Yes
Error handling	Yes

3. Test Environment

Element	Description
Operating system	Windows 11
Python version	Python 3.12
Communication protocol	TCP / UDP
Libraries used	socket, json, threading, network, etc.
Number of router applications	4
Controller IP address	127.0.0.1
Controller port	9000

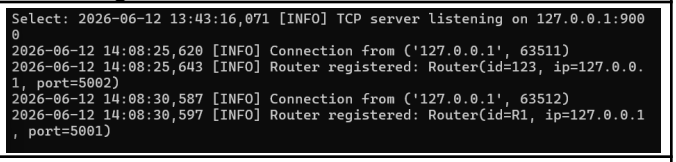
4. Test Case Template

Field	Description
Test case ID	TC-XX
Related requirement	FR-XX or NFR-XX

Test name	Short name of the test
Objective	What the test verifies
Preconditions	Conditions required before executing the test
Input data	Data used during the test
Test steps	Ordered steps to execute the test
Expected result	Correct system behavior
Actual result	Observed behavior
Status	Passed / Failed
Evidence	Screenshot, log, file, or terminal output
Observations	Additional comments

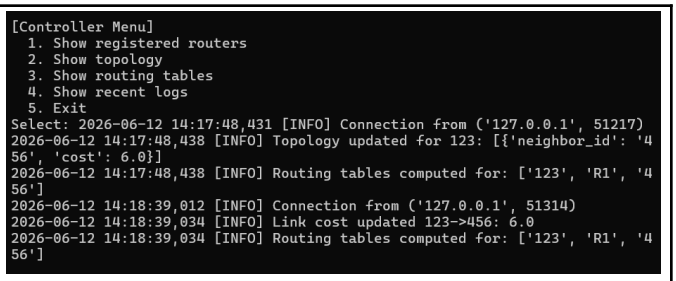
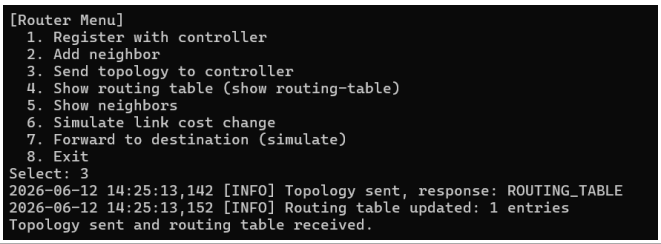
5. Functional Test Cases

TC-01: Router registration with the controller

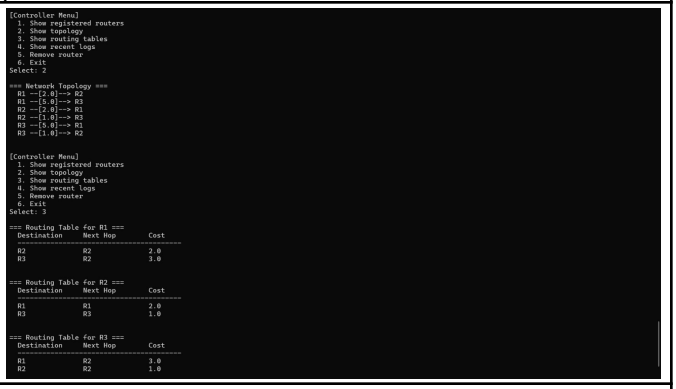
Field	Description
Test case ID	TC-01
Related requirement	FR-01
Test name	Router registration with the controller
Objective	Verify that a router can register successfully with the controller.
Preconditions	Controller application is running.
Input data	Router ID: R1, IP: 127.0.0.1, Port: 5001
Test steps	1. Start the controller. 2. Start router R1. 3. Send registration message. 4. Check controller registry.
Expected result	The controller stores R1 successfully and confirms registration.
Actual result	routers are being registered
Status	Router registered
Evidence	
Observations	none

TC-02: Send neighbor information to controller

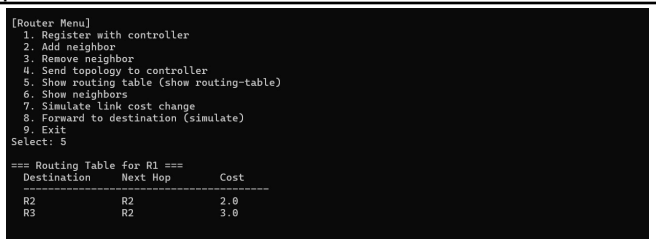
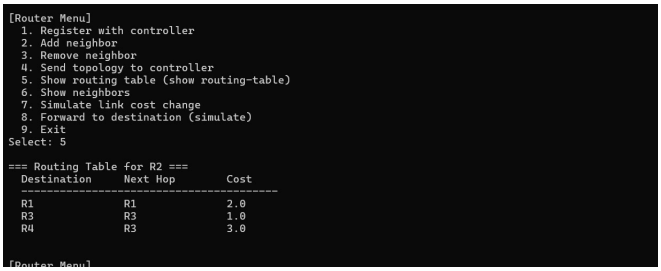
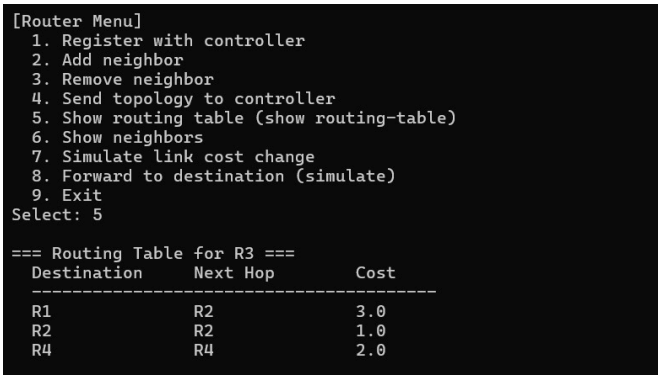
Field	Description
Test case ID	TC-02
Related requirement	FR-02, FR-03
Test name	Send neighbor information to controller
Objective	Verify that routers can send neighbor and link cost information.
Preconditions	Controller and routers are running.
Input data	R1 neighbors: R2 cost 10, R3 cost 5
Test steps	1. Start controller. 2. Start routers R1, R2, and R3. 3. Send neighbor information from R1. 4. Check topology stored in controller.
Expected result	The controller stores links R1-R2 and R1-R3 with their costs.
Actual result	pass

Evidence	 
Observations	none

TC-03: Dijkstra shortest path calculation

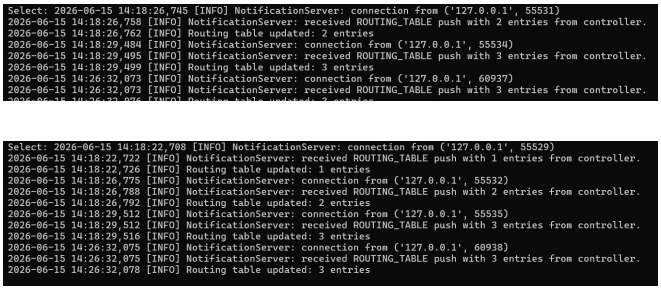
Field	Description
Test case ID	TC-03
Related requirement	FR-04
Test name	Dijkstra shortest path calculation
Objective	Verify that the controller computes the shortest path correctly.
Preconditions	Network topology is stored in the controller.
Input data	Topology: R1-R2 cost 2, R2-R3 cost 1, R1-R3 cost 5
Test steps	1. Load topology. 2. Execute Dijkstra from R1. 3. Check shortest path from R1 to R3.
Expected result	The shortest path from R1 to R3 is R1 -> R2 -> R3 with total cost 3.
Actual result	The controller accurately displays the network topology between routers R1, R2, and R3, including link costs. It also generates routing tables with the next hop and the calculated minimum cost for each destination.
Status	pass
Evidence	
Observations	The routing algorithm is verified to find the lowest-cost path. For example, to get from R1 to R3, R2 is used as the next hop with a total cost of 3.0 (2.0 + 1.0), instead of the direct link with a cost of 5.0. This behavior is consistent with the expected routing algorithm.

TC-04: Generate routing table for each router

Field	Description
Test case ID	TC-04
Related requirement	FR-05
Test name	Generate routing table for each router
Objective	Verify that the controller generates correct routing tables.
Preconditions	Topology is available and shortest paths were computed.
Input data	Network topology with at least four routers
Test steps	1. Load topology. 2. Execute routing algorithm. 3. Generate routing table for R1. 4. Verify destination, next hop, and cost.
Expected result	Routing table contains valid entries for all reachable destinations.
Actual result	The controller successfully generated routing tables for all registered routers based on the network topology and link costs. Each routing table contains the destination router, next hop, and total path cost.
Status	pass
Evidence	  
Observations	none

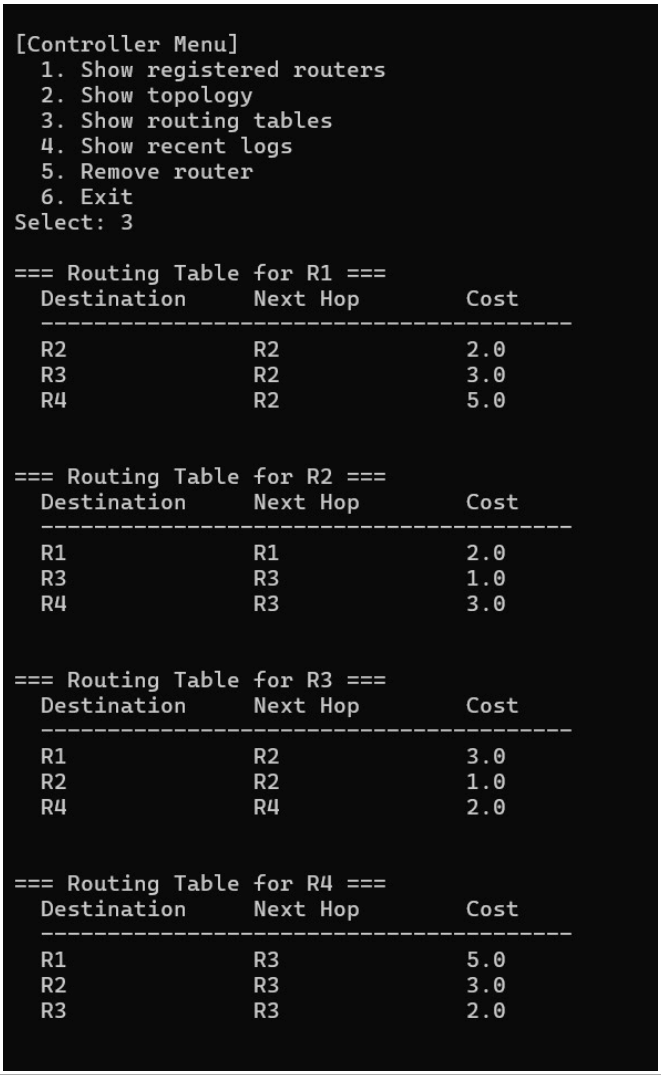
TC-05: Send routing table to router

Field	Description
Test case ID	TC-05
Related requirement	FR-06
Test name	Send routing table to router

Objective	Verify that the controller sends the correct routing table to each router.
Preconditions	Controller and router applications are running.
Input data	Routing table for R1
Test steps	1. Generate routing table in controller. 2. Send table to R1. 3. Check table received by R1.
Expected result	Router R1 receives and stores its routing table correctly.
Actual result	The controller successfully transmitted the generated routing tables to the corresponding routers. Each router received the routing information containing destinations, next hops, and path costs.
Status	pass
Evidence	
Observations	Communication between the controller and routers operated correctly. Routing tables were delivered without errors, and the received information matched the tables generated by the controller.

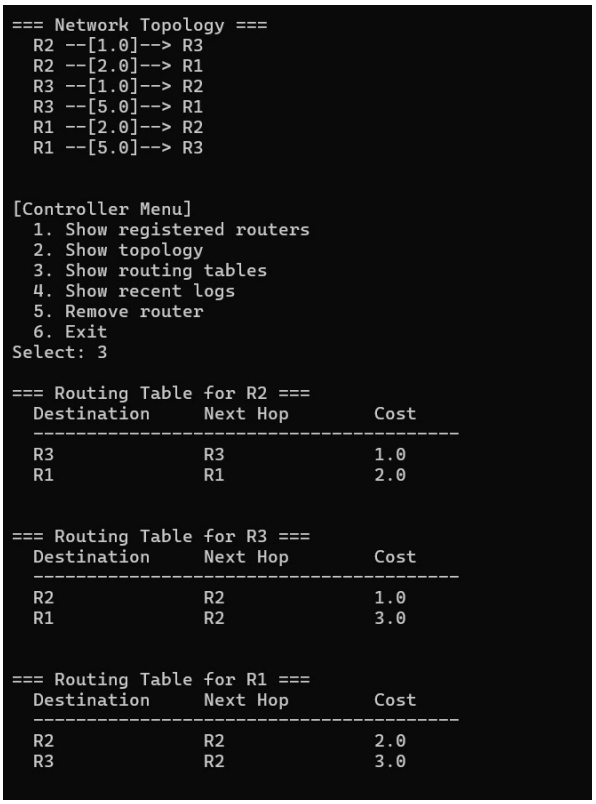
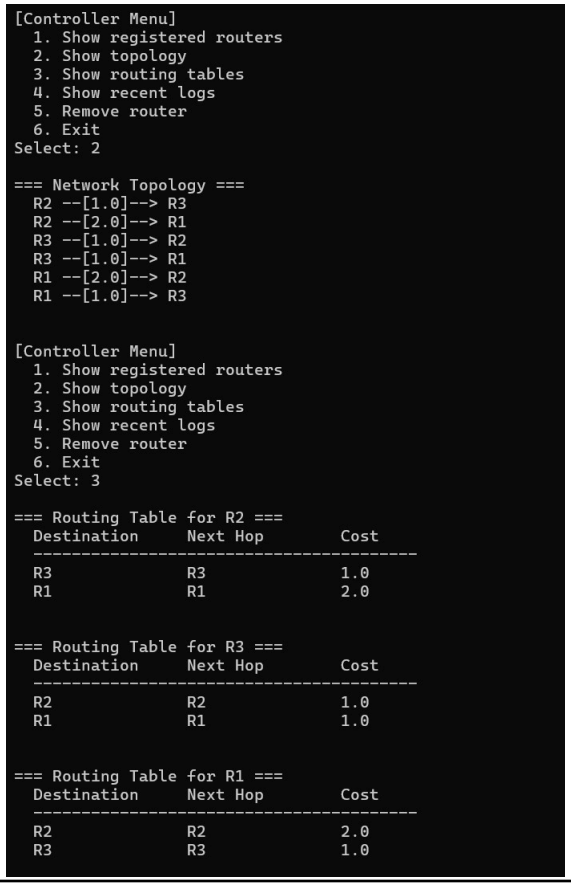
TC-06: Display routing table in CLI

Field	Description
Test case ID	TC-06
Related requirement	FR-07
Test name	Display routing table in CLI
Objective	Verify that the router displays the routing table in the terminal.
Preconditions	Router has already received a routing table.
Input data	Command: show routing-table
Test steps	1. Start router R1. 2. Receive routing table. 3. Execute display command.
Expected result	The routing table is displayed with destination, next hop, and cost.
Actual result	The controller successfully displayed the routing tables for all registered routers (R1, R2, and R3) through the command-line interface. Each table showed the destination router, next hop, and path cost.
Status	pass

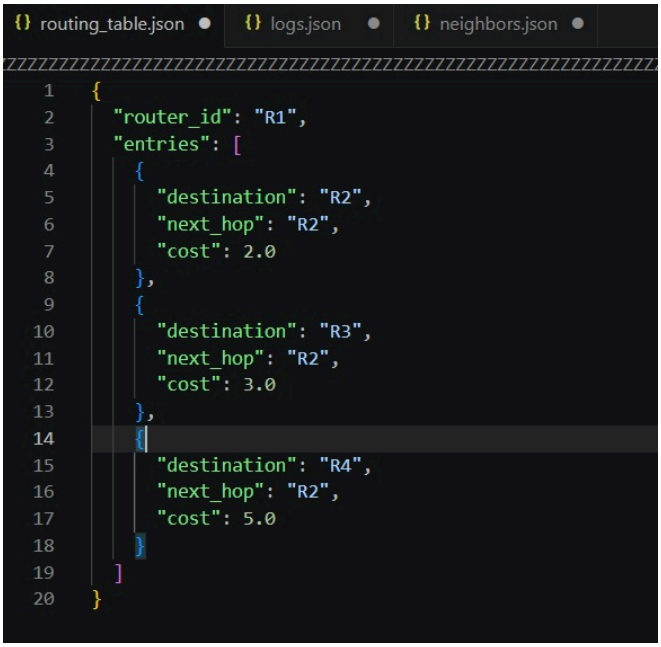
Evidence	
Observations	The displayed routing information is consistent with the network topology.

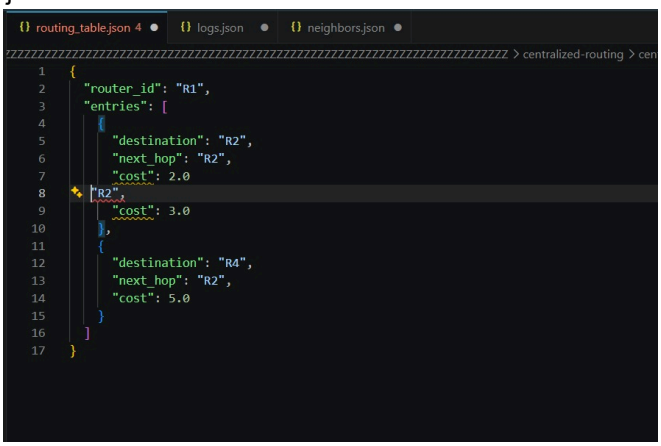
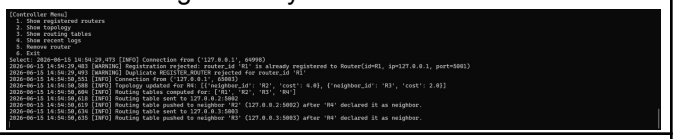
TC-07: Update link cost and recalculate routes

Field	Description
Test case ID	TC-07
Related requirement	FR-08
Test name	Update link cost and recalculate routes
Objective	Verify that the controller recalculates routes after a link cost change.
Preconditions	Initial topology and routing tables already exist.
Input data	Change cost R1-R3 from 5 to 1
Test steps	1. Load initial topology. 2. Generate initial routing tables. 3. Change link cost. 4. Recalculate routes. 5. Compare old and new routing table.
Expected result	Affected routing tables are updated according to the new shortest paths.
Actual result	The controller successfully updated the modified link cost, recalculated the shortest paths, and generated updated routing tables reflecting the new network conditions.

Status	pass
Evidence	 
Observations	none

TC-08: Invalid JSON message handling

Field	Description
Test case ID	TC-08
Related requirement	NFR-05
Test name	Invalid JSON message handling
Objective	Verify that the controller does not crash when it receives an invalid message.
Preconditions	Controller is running.
Input data	Malformed JSON message
Test steps	1. Start controller. 2. Send malformed JSON. 3. Observe controller behavior.
Expected result	The controller rejects the message and logs an error without crashing.
Actual result	The malformed JSON message was ignored by the controller. The application continued running normally and no interruption of service was observed.
Status	
Evidence	.json correct 

	.json incorrect  Code is working normally 
Observations	The controller demonstrated resilience against malformed input by maintaining normal operation. However, no explicit error message or log entry was generated, which could make debugging invalid messages more difficult. A future improvement would be to add detailed error logging for malformed JSON messages.

6. Requirements Traceability Matrix

Requirement ID	Description	Test case ID	Test status
FR-01	Router registration	TC-01	completed
FR-02	Neighbor information exchange	TC-02	completed
FR-03	Topology storage	TC-02	completed
FR-04	Shortest path calculation	TC-03	completed
FR-05	Routing table generation	TC-04	completed
FR-06	Routing table delivery	TC-05	completed
FR-07	Routing table display	TC-06	completed
FR-08	Link cost update	TC-07	completed
NFR-05	Invalid message handling	TC-08	completed

7. Test Execution Summary

Total test cases	Passed	Failed	Pending	Success percentage
8	8	0	0	100%

8. Final Test Report

Write a short paragraph describing the general result of the testing process, the main issues found, and the improvements required before final delivery.

Issue ID	Description	Severity	Related requirement	Suggested correction
ISS-01	The testing process was conducted to verify the functional and non-functional requirements of the centralized routing system. Most test cases were completed successfully, including router registration, neighbor information exchange, topology storage, routing table generation, routing table delivery, routing table display, link cost updates, and invalid message handling. The results indicate that the system is capable of maintaining network topology information and generating correct routing tables based on the available network data.	High / Medium / Low	none	none

03 System Architecture and Requirements Document Template

Student template for documenting the centralized routing system

Field	Description
Course	Informatics for Telecommunications
Program	Telecommunication Engineering
Project	Centralized Routing System
Team name	The developers.
Team members	Juan Jose Herrera, Jose David Cometa, Diana Isabel Chamorro
Professor	Oscar Mauricio Caicedo Rendon
Date	15/06/2026
Version	1.0

1. Introduction

1.1 Purpose of the Document

This document describes the requirements, architecture, components, communication mechanisms, data structures, and testing strategy of the centralized routing system developed as part of the Informatics for Telecommunications course.

1.2 Project Context

The project consists of developing a software-based centralized routing system in Python. The system includes a controller application and several router applications. The controller collects topology information, computes routing paths, generates routing tables, and distributes them to routers.

1.3 Project Scope

Included	Not included
Controller application	Real physical routers
Router applications	Real packet forwarding
TCP or UDP communication	Internet-scale routing
JSON-based messages	Full BGP, OSPF, or RIP implementation
Shortest path calculation	Hardware-level routing
Routing table generation	Production-grade SDN controller

2. Objectives

General objective: Develop a centralized routing system in Python that allows router applications to register with a controller, send topology information, receive routing tables, and simulate routing decisions based on shortest path computation.

1. Design the architecture of a centralized routing system composed of a controller and multiple router applications.
2. Implement communication between routers and the controller using sockets and JSON messages.
3. Implement a routing algorithm to compute shortest paths and generate routing tables.
4. Validate the system through functional and non-functional test cases.

3. System Overview

The centralized routing system follows a controller-based architecture. Router applications do not compute complete network routes independently. Each router sends its local information to the controller. The controller builds a global view of the network and computes routing tables for all routers.

Component	Description
Controller application	Central entity that manages topology, computes routes, and sends routing tables
Router application	Software entity that registers with the controller and receives routing information
Communication module	Handles message exchange between controller and routers
Topology manager	Stores and updates the network topology
Routing algorithm module	Computes shortest paths using Dijkstra algorithm
Routing table manager	Generates and stores routing tables
CLI module	Allows users to interact with the system through the command line
Logging module	Records relevant system events

4. Functional Requirements

ID	Requirement	Description	Priority
FR-01	Router registration	Routers must register with the controller by sending router ID, IP address, and port.	High
FR-02	Neighbor information exchange	Routers must send neighbor and link cost information to the controller.	High
FR-03	Topology storage	The controller must store the network topology.	High
FR-04	Route computation	The controller must compute shortest paths using Dijkstra algorithm.	High

FR-05	Routing table generation	The controller must generate a routing table for each router.	High
FR-06	Routing table delivery	The controller must send the corresponding routing table to each router.	High
FR-07	Routing table visualization	Routers must display their routing tables through the command line.	Medium
FR-08	Link cost update	The system should allow link cost updates and route recalculation.	Medium
FR-09	Event logging	The system should log registrations, topology updates, and routing table delivery.	Low

5. Non-Functional Requirements

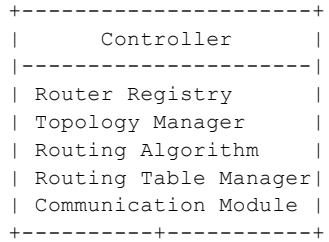
ID	Requirement	Description	Priority
NFR-01	Python implementation	The system must be implemented in Python.	High
NFR-02	Modular design	The code must be organized into clear modules or classes.	High
NFR-03	JSON messages	The system must use JSON for message exchange.	High
NFR-04	Command-line interface	The system must be operated from the terminal.	High
NFR-05	Error handling	The system must handle invalid messages without crashing.	Medium
NFR-06	Scalability	The system should support at least four routers.	Medium
NFR-07	Maintainability	The code should be readable and documented.	Medium
NFR-08	Testability	The system should include documented test cases.	Medium

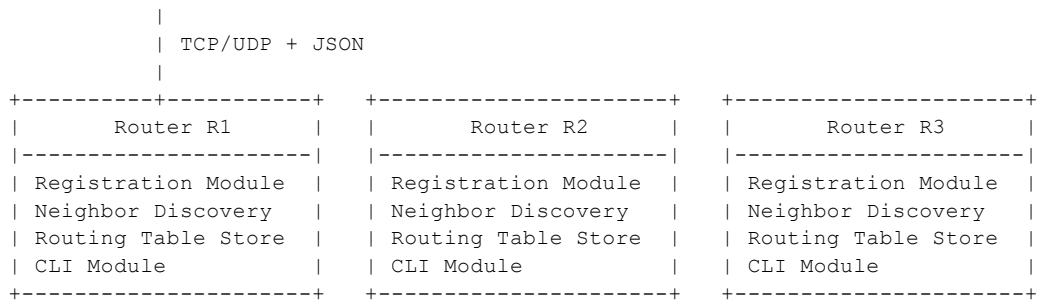
6. System Architecture

6.1 Architectural Style

The system follows a centralized controller-based architecture, similar to the basic idea of Software-Defined Networking at a simplified educational level.

6.2 Architecture Diagram





7. Component Description

Component	Main responsibility	Input	Output
Controller application	Manage global network information and compute routes	Router registration, topology updates, link cost updates	Routing tables, ACK messages, error messages
Router application	Represent a network router in software	Routing table from controller	Registration message, neighbor information, link cost updates
Topology manager	Store the network topology	Nodes, links, costs	Graph or adjacency list
Routing algorithm module	Compute shortest paths	Network topology	Paths and costs
Routing table manager	Convert paths into routing table entries	Shortest paths	Destination, next hop, cost entries

8. Communication Design

Field	Description
Protocol	TCP or UDP
Data format	JSON
Encoding	UTF-8
Communication model	Client-server
Server	Controller
Clients	Router applications

Message type	Sender	Receiver	Purpose
REGISTER_ROUTER	Router	Controller	Register a router
TOPOLOGY_UPDATE	Router	Controller	Send neighbor information
ROUTING_TABLE	Controller	Router	Send routing table
LINK_COST_UPDATE	Router/Admin	Controller	Update link cost
ERROR	Controller/Router	Controller/Router	Report an error
ACK	Controller/Router	Controller/Router	Confirm successful operation

9. Example JSON Messages

Router registration message:

```
{
  "type": "REGISTER_ROUTER",
  "router_id": "R1",
  "ip": "127.0.0.1",
  "port": 5001
}
```

Topology update message:

```
{
  "type": "TOPOLOGY_UPDATE",
  "router_id": "R1",
  "neighbors": [
    {"neighbor_id": "R2", "cost": 10},
    {"neighbor_id": "R3", "cost": 5}
  ]
}
```

Routing table message:

```
{
  "type": "ROUTING_TABLE",
  "router_id": "R1",
  "routing_table": [
    {"destination": "R2", "next_hop": "R2", "cost": 10},
    {"destination": "R4", "next_hop": "R3", "cost": 8}
  ]
}
```

10. Data Model

Entity	Field	Type	Description
Router	router_id	String	Unique router identifier
Router	ip	String	Router IP address
Router	port	Integer	Router communication port
Router	status	String	Active or inactive
Link	source_router	String	Source router ID
Link	destination_router	String	Destination router ID
Link	cost	Integer/Float	Link cost
Routing table entry	destination	String	Destination router
Routing table entry	next_hop	String	Next router in the path
Routing table entry	cost	Integer/Float	Total path cost

11. Suggested Projects Structure

CONTROLLER

```
centralized-routing-controller/
|-- app/
|   |-- main.py
|   |-- controllers/
|   |   |-- controller_app_controller.py
|   |   |-- router_registry_controller.py
|   |   |-- topology_controller.py
|   |   |-- routing_controller.py
|   |   `-- communication_controller.py
|   |-- models/
|   |   |-- router.py
|   |   |-- link.py
|   |   |-- topology.py
|   |   |-- routing_table.py
|   |   `-- routing_table_entry.py
```

```
|   |-- views/
|   |   |-- controller_cli_view.py
|   |   |-- topology_view.py
|   |   |-- routing_table_view.py
|   |   `-- log_view.py
|   |-- dao/
|   |   |-- router_dao.py
|   |   |-- topology_dao.py
|   |   |-- routing_table_dao.py
|   |   `-- log_dao.py
|   |-- services/
|   |   |-- dijkstra_service.py
|   |   |-- routing_table_service.py
|   |   |-- topology_service.py
|   |   `-- message_service.py
|   |-- network/
|   |   |-- tcp_server.py
|   |   |-- udp_server.py
|   |   `-- client_handler.py
|   |-- config/
|   |   |-- controller_config.json
|   |   `-- logging_config.json
|   |-- data/
|   |   |-- routers.json
|   |   |-- topology.json
|   |   |-- routing_tables.json
|   |   `-- logs.json
|   `-- utils/
|       |-- constants.py
|       |-- logger.py
|       `-- json_utils.py|
|-- tests/
|   |-- test_router_registry.py
|   |-- test_topology.py
|   |-- test_dijkstra_service.py
|   |-- test_routing_table_service.py
|   `-- test_controller_communication.py
|-- docs/
|   |-- requirements.md
|   |-- architecture.md
|   |-- api_messages.md
|   `-- test_plan.md
```

```
|-- README.md
`-- requirements.txt
```

ROUTER

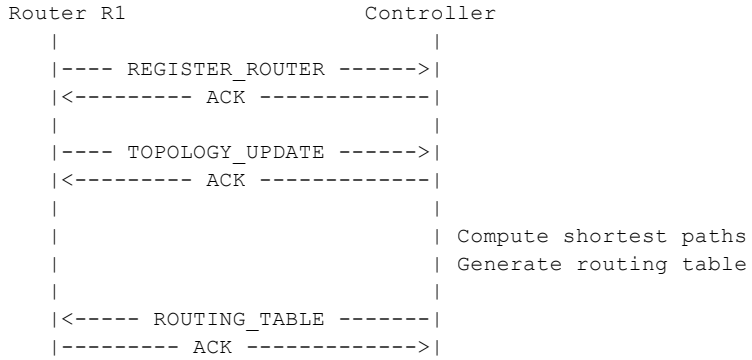
```
centralized-routing-router/
|-- app/
|   |-- main.py
|   |-- controllers/
|   |   |-- router_app_controller.py
|   |   |-- registration_controller.py
|   |   |-- neighbor_controller.py
|   |   |-- routing_table_controller.py
|   |   `-- forwarding_controller.py
|   |-- models/
|   |   |-- router.py
|   |   |-- neighbor.py
|   |   |-- link.py
|   |   |-- routing_table.py
|   |   `-- routing_table_entry.py
|   |-- views/
|   |   |-- router_cli_view.py
|   |   |-- routing_table_view.py
|   |   |-- neighbor_view.py
|   |   `-- forwarding_view.py
|   |-- dao/
|   |   |-- router_config_dao.py
|   |   |-- neighbor_dao.py
|   |   |-- routing_table_dao.py
|   |   `-- log_dao.py
|   |-- services/
|   |   |-- registration_service.py
|   |   |-- neighbor_service.py
|   |   |-- routing_table_service.py
|   |   |-- forwarding_service.py
|   |   `-- message_service.py
|   |-- network/
|   |   |-- tcp_client.py
|   |   |-- udp_client.py
|   |   `-- controller_connection.py
|   |-- config/
```

```
|   |   |-- router_config.json
|   |   `-- controller_config.json
|   |-- data/
|   |   |-- neighbors.json
|   |   |-- routing_table.json
|   |   `-- logs.json
|   `-- utils/
|       |-- constants.py
|       |-- logger.py
|       `-- json_utils.py
|-- tests/
|   |-- test_registration.py
|   |-- test_neighbor_service.py
|   |-- test_routing_table_storage.py
|   |-- test_forwarding_service.py
|   `-- test_router_communication.py
|-- docs/
|   |-- requirements.md
|   |-- architecture.md
|   |-- api_messages.md
|   `-- test_plan.md
|-- README.md
`-- requirements.txt
```

12. Routing Algorithm Description

Students must explain the selected algorithm. For example: The system uses Dijkstra algorithm to compute the shortest path from each router to all other routers. The controller applies the algorithm over the global network topology. After computing the shortest paths, the controller extracts the next hop for each destination and builds a routing table for every router.

13. Sequence Diagram



14. Test Strategy

Test area	Description
Router registration	Verify that routers register correctly
Topology update	Verify that topology information is received and stored
Shortest path calculation	Verify that the routing algorithm returns correct paths
Routing table generation	Verify destination, next hop, and cost
Routing table delivery	Verify that each router receives its table
Error handling	Verify invalid messages do not crash the system

15. Conclusions

Students should answer: Was the system implemented successfully? Did the controller correctly compute routing tables? Did the routers receive and display routing information? What were the main technical challenges? What improvements could be added in future versions?

The system was implemented successfully and achieved the main objectives defined in the project requirements. The centralized controller was able to register routers, collect neighbor information, store the network topology, and generate routing tables based on the available link costs.

The controller correctly computed routing tables, selecting the lowest-cost paths between routers.

The routers successfully received and displayed routing information delivered by the controller. Routing tables were updated and presented correctly, allowing each router to identify the appropriate next hop and path cost for reaching other destinations in the network.

The main technical challenges included designing the communication protocol between routers and the controller, maintaining a consistent topology database, handling asynchronous network messages, and ensuring that routing tables were updated correctly whenever topology information changed.

Future versions of the system could include automatic topology visualization, support for larger and more complex networks, dynamic failure detection and recovery, route recalculation in real time, graphical user interfaces for monitoring, persistent database storage instead of JSON files, and enhanced security mechanisms for message authentication and validation.

Overall, the project demonstrates a functional centralized routing solution that satisfies the majority of the specified requirements and provides a solid foundation for future enhancements.